

A 28-nm Digital Transpose SRAM Compute-in-Memory Macro With Accurate/Approximate Dual Mode for Floating-Point Edge Training and Inference

Yiyang Yuan¹, Bingxin Zhang, Yiming Yang, Yishan Luo, Qirui Chen², Haitao Wang, Qihao Liu³, Zhiming Chen⁴, Hao Wu⁵, Jinshan Yue⁶, *Member, IEEE*, Shidong Lv, Xinghua Wang⁷, Pui-In Mak⁸, *Fellow, IEEE*, Xiaoran Li⁹, and Feng Zhang¹⁰, *Senior Member, IEEE*

Abstract—Static random-access memory (SRAM)-based computing-in-memory (CIM) macros have been widely studied to improve the energy efficiency of edge artificial intelligence (AI) inference tasks. However, less attention has been given to AI training, which requires CIM macros to not only perform matrix multiply-accumulate (MAC) operations but also support matrix transposition. To address the limitations of previous analog transpose and digital non-transpose SRAM CIM macros, this work features: 1) a cyclic-weight-mapping SRAM array that enables matrix transposition and reuse of MAC circuits during both feed-forward (FF) and back-propagation (BP) phases; 2) a digital CIM architecture employing signed fixed-point mantissa encode and a vector-wise pre-alignment (VWPA) scheme, supporting multiple data formats including INT4/8, FP8, and BF16; and 3) an accurate/approximate dual-mode bit-parallel MAC circuit (DMBP-MAC) designed to provide a tradeoff between computational accuracy and energy efficiency. A fabricated 28-nm 32-kB transpose SRAM CIM macro achieved average energy efficiency of 70.2–285.4 TOPS/W in INT4, 17.5–71.4 TOPS/W in INT8, 51.1–192.3 TFLOPS/W in FP8, and 12.8–48 TFLOPS/W in BF16.

Index Terms—Approximate computing, artificial intelligence (AI), computing-in-memory (CIM), floating point (FP), static random-access memory (SRAM).

I. INTRODUCTION

WITH the rapid development of artificial intelligence (AI) neural network (NN) models, AI-specific hardware has been increasingly integrated into edge devices. However, the massive data transfer between computation units and memory has become a major bottleneck in terms of both energy efficiency and throughput, especially for power-constrained edge devices. To address this challenge, computing-in-memory (CIM) techniques have been proposed as a promising solution by tightly integrating computing units with memory cells, significantly improving the energy efficiency of AI hardware [1], [2], [3], [4].

Most CIM studies have focused on NN inference [5], [6], [7], [8], [9], [10], with significantly less attention given to the training process. However, fine-tuning training is a critical phase in edge NN model deployment, as it enhances pre-trained model performance by learning patterns from user data and real-world data, which enables pre-trained models to deliver personalized services and adapt to real-world environmental changes [11]. As illustrated in Fig. 1(a) and (b), compared to training NN models on cloud servers, edge-based training offers advantages such as lower latency, reduced power consumption, and enhanced privacy protection by avoiding the need to upload local data to remote servers.

In NN model training, the two fundamental phases are feed-forward (FF) computation and back-propagation (BP). As illustrated in Fig. 1(c), the FF phase is identical to that of inference, while the BP phase involves multiplying the error gradient with the transposed weight matrix. Several studies have proposed transpose-enabled CIM macros to support both FF and BP operations [12], [13]. Furthermore, on-chip training architectures based on such transpose CIM macros have also been explored [14]. However, the training phase is highly sensitive to accuracy loss, which is markedly different from the inference phase. Most previous transpose CIM macros are based on analog CIM schemes, which are susceptible to process, voltage, and temperature (PVT) variations. Moreover,

Received 29 June 2025; revised 30 November 2025 and 12 February 2026; accepted 24 March 2026. Date of publication 7 April 2026; date of current version 27 August 2026. This article was approved by Associate Editor Ken Takeuchi. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB4402400; in part by the National Natural Science Foundation of China under Grant 92464201, Grant U2341218, and Grant 6232241; and in part by Beijing Natural Science Foundation under Grant 4262087 and Grant MIIT TC230A076-1. (Corresponding authors: Xiaoran Li; Xinghua Wang; Feng Zhang.)

Yiyang Yuan, Bingxin Zhang, Yishan Luo, Hao Wu, Jinshan Yue, and Feng Zhang are with the Institute of Microelectronics of Chinese Academy of Sciences (IMECAS), Beijing 100029, China (e-mail: zhangfeng_ime@ime.ac.cn).

Yiming Yang, Qirui Chen, Zhiming Chen, Shidong Lv, and Xiaoran Li are with the School of Integrated Circuits and Electronics, Beijing Institute of Technology, Beijing 100081, China (e-mail: xiaoran.li@bit.edu.cn).

Haitao Wang is with Chongqing Institute of Microelectronics and Microsystems, Beijing Institute of Technology (BIT), Chongqing 401332, China.

Qihao Liu is with Shandong Sino-Chip Semiconductor Company Ltd., Jinan 250101, China.

Xinghua Wang is with the School of Integrated Circuits and Electronics, Beijing Institute of Technology, Beijing 100081, China, also with Chongqing Institute of Microelectronics and Microsystems, Beijing Institute of Technology, Chongqing 401332, China, also with the Advanced Technology Research Institute, Beijing Institute of Technology, Jinan 250101, China, and also with the Yangtze Delta Region Academy, Beijing Institute of Technology, Jiaxing 314000, China (e-mail: 89811@bit.edu.cn).

Pui-In Mak is with the University of Macau (UM), Macau, China. Digital Object Identifier 10.1109/JSSC.2026.3679560

0018-9200 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: Beijing Normal University. Downloaded on September 08, 2026 at 07:42:15 UTC from IEEE Xplore. Restrictions apply.

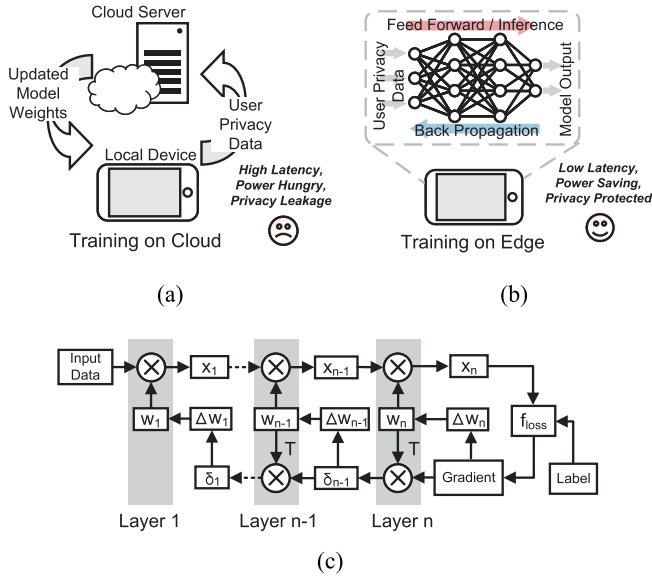


Fig. 1. (a) Remote training in cloud sever. (b) Local training in edge device. (c) Training computation dataflow.

the data formats supported by these macros are often limited to integers (INTs), which can lead to instability during the training process.

This work proposes a 28-nm digital 6T bit-cell floating-point (FP) static random-access memory (SRAM) transpose CIM macro. The proposed CIM macro incorporates a cyclic-weight-mapping SRAM array (CWM-SRAM), a signed-fixed mantissa encode (SFME) scheme, a vector-wise pre-alignment (VWPA) mechanism, and dual-mode bit-parallel MAC circuits (DMBP-MACs).

The remainder of this article is organized as follows. Section II describes the challenges of currently existing transpose SRAM CIM macros and our motivations. Section III outlines the overall structure of the proposed transpose SRAM CIM macro and discusses the CWM-SRAM scheme. Section IV describes the proposed SFME and VWPA. Section V describes the DMBP-MAC. Section VI describes the analysis and measured results of the proposed CIM macro. Section VII gives this work a final conclusion.

II. CHALLENGES AND MOTIVATIONS

A. Separated Data Paths and Redundant MAC Circuits

To support matrix transposition, previous transpose-enabled CIM macros employ separate data paths and multiply-accumulate (MAC) circuits for FF and BP. A representative transpose CIM macro architecture [12] is illustrated in Fig. 2. In the FF dataflow, activations are transmitted along the word line (WL) direction. After being multiplied with the weights stored in SRAM cells within local compute units, the resulting partial sums are accumulated and sent to the FF MAC circuit, which consists of analog-to-digital converters (ADCs) and digital accumulators, along the bitline (BL) direction. In contrast, during the BP dataflow, error gradients are input along the BL direction, and the computed partial sums are accumulated and routed to the BP MAC circuit along the WL direction.

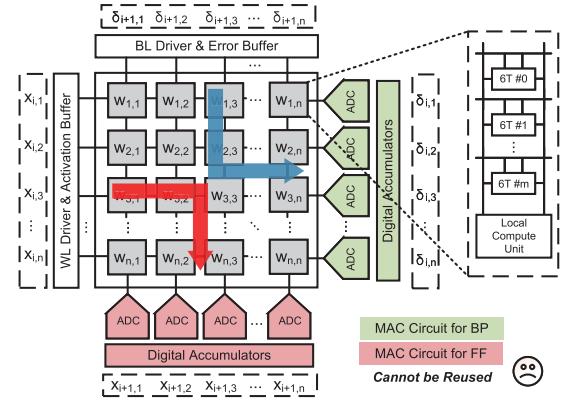


Fig. 2. Separated FF and BP data paths and redundant MAC circuits in previous transpose CIM macros.

The separated data paths effectively prevent interference between dataflows in FF and BP phases. However, this approach introduces redundancy in the MAC circuits, which occupy a significant portion of the power and area in CIM macros. In addition, since the local compute units are shared between FF and BP operations, the two phases cannot be executed concurrently. As a result, the redundant MAC circuits lead to substantial power and area overheads, significantly degrading both energy efficiency and area efficiency in such transpose CIM macros. Venkataramanaiah et al. [15] propose a cyclic weight storage and access scheme that enables both FF and BP to share a single data port. However, directly applying this approach to CIM architectures faces several critical challenges. First, the work lacks a concrete circuit-level design methodology: implementing diagonal-parallel readout is not feasible on a conventional SRAM array without significant modifications to peripheral circuitry. Second, it does not address how the mapping scheme should adapt to variable-precision operands, which is a capability essential for unified training and inference support in edge AI devices. Finally, the alignment in [15] is performed on the weight side via rotation during readout, which introduces redundant alignment overhead in activation-broadcast CIM architectures, where shifting activations instead of weights is far more efficient due to their broadcast nature.

To address these challenges, this work proposes a detailed CWM-SRAM design that enables reuse of the data paths in both FF and BP for transpose CIM. As a result, the MAC circuits can be shared between the two phases, thereby achieving higher energy and area efficiency.

B. Restricted Data Format Supporting

Compared to inference, training is much more sensitive to reduced computing precision. As shown in Fig. 3(a), studies indicate that using simple uniform INT8 quantization in training leads to a significant accuracy drop and causes training to fail after a certain number of iterations. This is because the INT format cannot adapt to the dynamic and varying nature of error gradient distributions [16]. Although complex nonuniform INT quantization schemes may allow successful

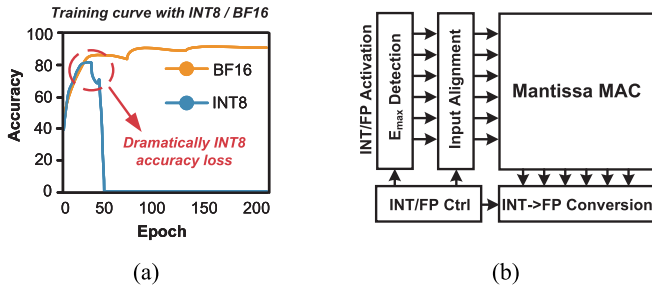


Fig. 3. (a) Training accuracy loss with INT/FP format. (b) Structure of pre-alignment FP CIM macro.

training, they require additional design efforts and hardware overheads to account for gradient distribution characteristics.

This issue has been largely overlooked in previous transpose-enabled CIM macros. Fortunately, FP CIM schemes have seen significant development in recent years [5], [17], [18], [19], [20]. These FP CIMs perform offline alignment of weight mantissas and are therefore referred to as pre-alignment FP CIMs. As shown in Fig. 3(b), a typical pre-alignment FP CIM first performs online activation mantissa alignment using a maximum exponent detection circuit and a mantissa alignment circuit. The aligned activation mantissas, together with the pre-aligned weight mantissas, then undergo MAC operations within an INT CIM array. Finally, the INT-format output is reformatted into FP format through a format conversion circuit. Although pre-alignment FP CIMs support FP format, the significant accuracy loss caused by mantissa truncation in coarse-grained pre-alignment [20] renders them unsuitable for accuracy-sensitive training processes.

These challenges motivate our work to integrate FP CIM techniques into the transpose CIM macro using SFME, thereby enabling native support for FP format. Additionally, we employ VWPA to achieve fine-grained alignment and further improve computational accuracy.

C. Imbalance Between Memory and Computing

Previous analog-domain transpose CIM macros are susceptible to PVT variations, which compromises their stability and accuracy during the training process. In recent years, digital-domain CIM macros have been developed to enable stable and fully accurate MAC operations [6], [21], [22], [23], [24].

Although digital-domain CIM macros overcome the limitations caused by PVT variations, achieving a balance between memory density and area efficiency remains a key challenge [5], [6]. Allocating more area to MAC circuits results in higher area efficiency but lower memory density, whereas allocating more area to SRAM cells leads to higher memory density but reduced area efficiency. Although improving area efficiency can enhance computing throughput, reduced memory density may result in lower memory capacity and data reusability, which increases data movement frequency and ultimately degrades overall system energy efficiency.

To achieve a balanced tradeoff among area efficiency, energy efficiency, and memory density in the proposed digital-domain CIM macro, the DWBP-MAC is introduced to reduce

the number of computing cycles and improve throughput through a bit-parallel CIM architecture. Additionally, area and energy efficiency are further enhanced by incorporating an approximate computing scheme. To improve memory density, the number of rows in the SRAM array is increased and a centrosymmetric layout is implemented to improve area utilization.

III. CYCLIC-WEIGHT-MAPPING TRANSPOSE CIM MACRO

A. Macro Overall Structure

Fig. 4(a) illustrates the architecture of our proposed digital transpose FP CIM macro, which comprises four key components: 1) a read/write control circuit; 2) an activation rotate-and-align circuit; 3) a bus interface circuit; and 4) two sets of CWM-SRAM and DMBP-MAC arranged in an upper and lower block configuration. For the purpose of facilitating seamless system integration, the macro is engineered to be compliant with the standard advanced microcontroller bus architecture (AMBA). The advanced high-performance bus (AHB) interface is designed to manage high bandwidth memory accesses, whereas the advanced peripheral bus (APB) interface is set up to convey control signals, activation vectors, and MAC results.

The macro configurations for INT4/8, FP8, and BF16 formats are depicted in Fig. 4(b). In the INT4 mode, the upper and lower blocks store two distinct sets of 4-bit weight elements. Conversely, in the INT8 mode, an 8-bit weight element is partitioned into 4 most significant bits (MSBs) and 4 least significant bits (LSBs), which are then stored in the upper and lower blocks, respectively. Both the FP8 and BF16 modes are designed to leverage the data paths used in the INT4 and INT8 modes. In the FP8 mode, a 4-bit aligned mantissa is stored to represent an FP8 weight. As such, the upper and lower blocks can accommodate two distinct sets of 4-bit FP8 mantissas. For the BF16 mode, an 8-bit aligned mantissa is stored after being partitioned into 4 MSBs and 4 LSBs, similar to the approach in the INT8 mode.

Fig. 4(c) presents the timing diagram for different data formats. The macro is engineered to process two 64-channel 4-bit MAC operations within a single cycle. Consequently, in both the INT4 and FP8 modes, the MAC operation can be completed in just one MAC cycle. In contrast, for the INT8 and BF16 modes, two MAC cycles are necessary to finish the computation. During these cycles, the activation is sent into the macro in a segmented manner: the 4 MSBs are fed in during the first cycle and the 4 LSBs are fed in during the second cycle.

B. Cyclic-Weight-Mapping SRAM Array

Fig. 5(a) illustrates the architecture of the proposed CWM-SRAM array. The array, configured with 64 rows and 256 columns, is partitioned into 64 sub-banks, each comprising 64 rows and 4 columns. Within this architecture, every set of four adjacent column SRAM cells forms a fundamental 4-bit storage unit for weight representation. Consequently, the CWM-SRAM array effectively constitutes a 64×64 array of 4-bit cells. The row selectors, driven by 5-bit address signals,

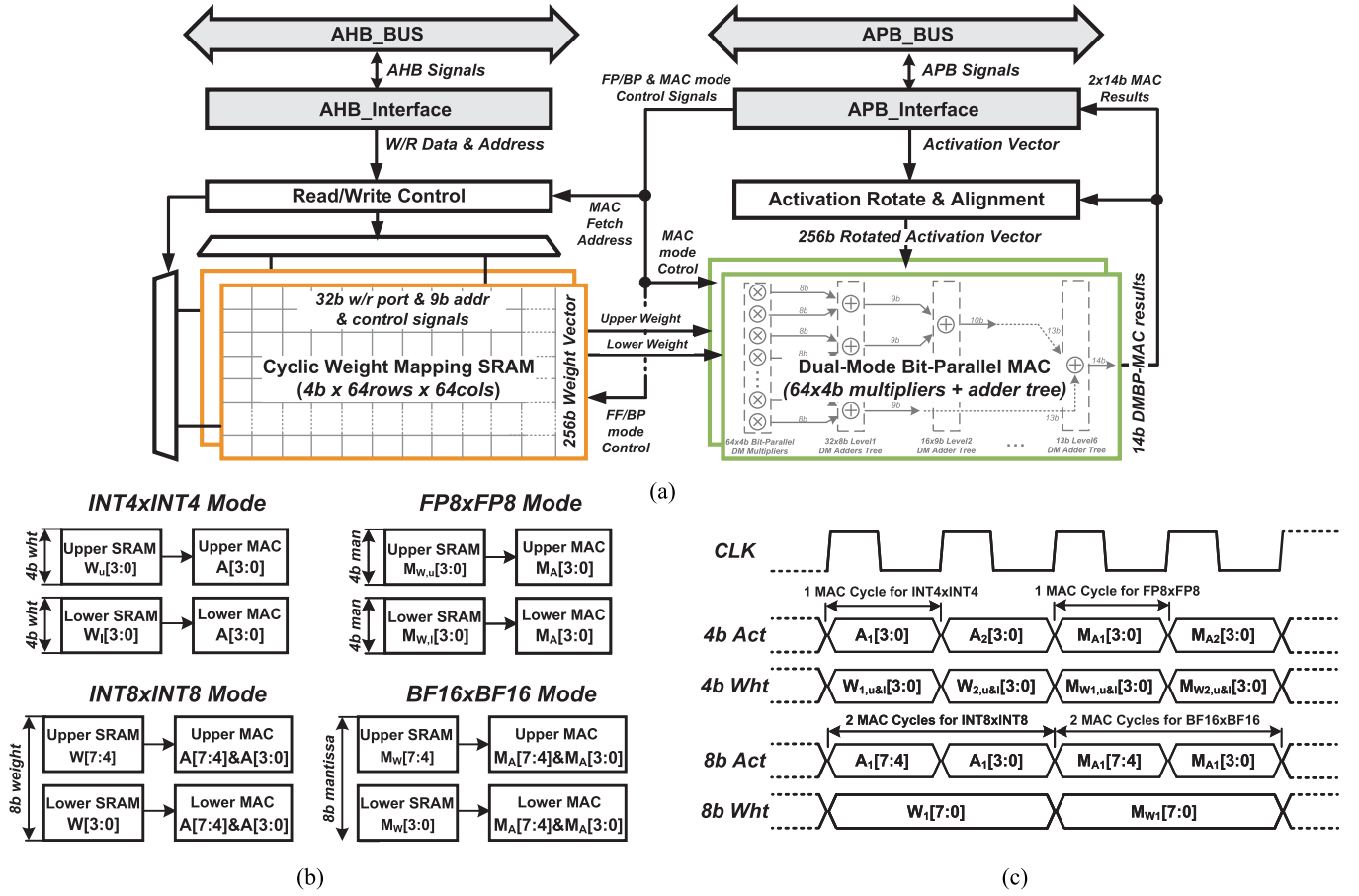


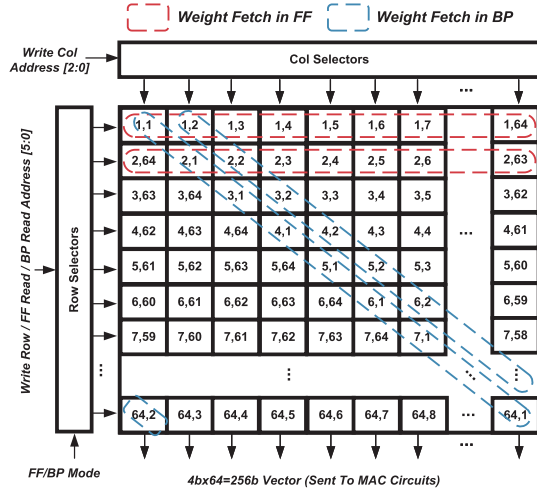
Fig. 4. (a) Overall architecture of the proposed digital transpose FP CIM macro. (b) CIM macro configuration for INT4/8, FP8, and BF16 formats. (c) Timing diagrams for INT4/8, FP8, and BF16 formats.

are capable of selecting 64 4-bit cells simultaneously. During normal read/write operations, the column selectors, controlled by 2-bit address signals, further select 8 out of the 64 4-bit cells selected by the same row signal to support a standard 32-bit read/write data bus width. During MAC operations, the row selectors are driven by a 5-bit FF/BP address and a mode control signal that determines whether the circuit operates in FF or BP mode. In the FF mode, the row address is similar to the standard row select signal used in normal read/write operations, which activates all 4-bit cells within the same row. In contrast, in BP mode, the BP address is used to select 4-bit cells along a corresponding diagonal direction, enabling the transpose computation required for gradient propagation. Notably, the column selectors remain inactive in this scenario. A 256-bit vector is ultimately delivered to the MAC circuits in both FF and BP modes, serving as the input for parallel MAC operations.

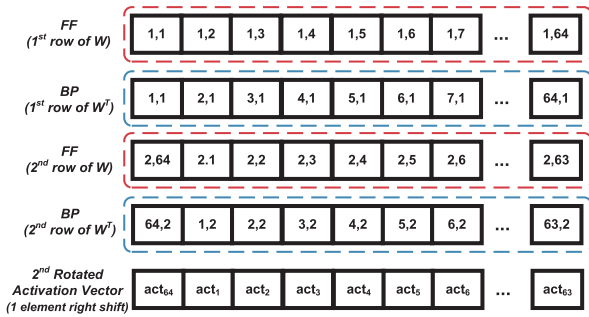
To enable the transpose operation, the weight matrix is stored using a cyclic-weight-mapping scheme, as illustrated in Fig. 5(b). In this scheme, each row of the matrix is right-shifted by a number of elements corresponding to its row index. For instance, the second row is shifted by one element, and the third row by two elements. In this way, during FF mode, row vectors of the matrix can be directly accessed. Meanwhile, in BP mode, the elements along a given

diagonal direction correspond exactly to the column vectors of the original weight matrix. To maintain alignment with the right-shifted weight vectors, the activation vector is also right-shifted within the rotate-and-align circuit. This ensures that corresponding elements of the activation and weight vectors are spatially aligned for accurate MAC operations in both FF and BP modes.

As shown in Fig. 6(a), the four SRAM cells within a 4-bit cell can be activated by the FF or BP row select read enable signals. The corresponding timing diagram of the 4-bit cell is presented in Fig. 6(b). Overall, the CWM-SRAM array follows a standard SRAM read operation: precharging the BLs, enabling the WLs, which is controlled by the FF or BP row select read enable signals, disabling the WLs, and then sensing the final data via the sense amplifier. During computation, the read data are kept to enable repeated use within the compute cycle, thereby avoiding redundant read operations and minimizing energy overhead. In the actual circuit implementation, the row select enable signals are synchronized with the system clock and generated by the row decoder; sufficient timing margin is guaranteed through careful timing analysis across PVT corners. This scheme incurs a 46.5% area overhead for the SRAM array, primarily due to the additional area required for the control gates and the increased spacing margins needed for metal routing. The diagonal interconnections required in

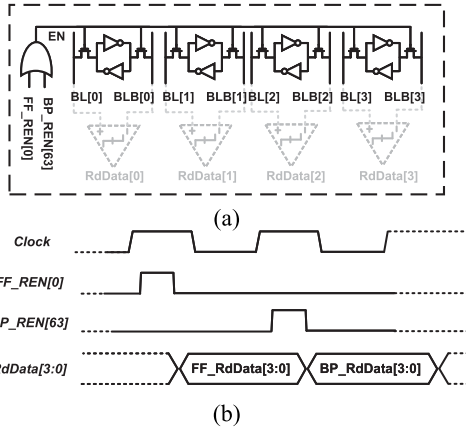


(a)



(b)

Fig. 5. (a) Structure of CWM-SRAM. (b) Row and column vectors stored with cyclic weight mapping.



(b)

Fig. 6. (a) Single 4-bit cell with write signals and circuits ignored. (b) Timing diagram of the 4-bit cell.

BP mode are implemented using two dedicated metal layers. Specifically, the second and third metal layer (M2 and M3) is utilized to connect 4-bit cells within the same diagonal, while the fourth metal layer (M4) is employed to interconnect the corresponding diagonals. To avoid routing congestion with M4, the FF signals are routed in the fifth metal layer (M5). An example layout of a 4×4 array composed of 4-bit cells is illustrated in Fig. 7.

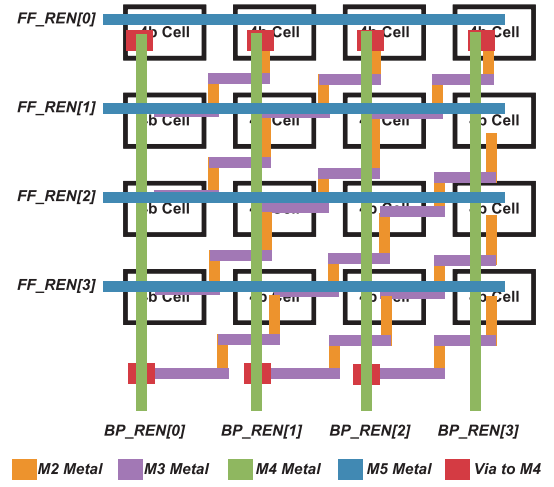


Fig. 7. Example layout of a 4×4 4-bit cells array.

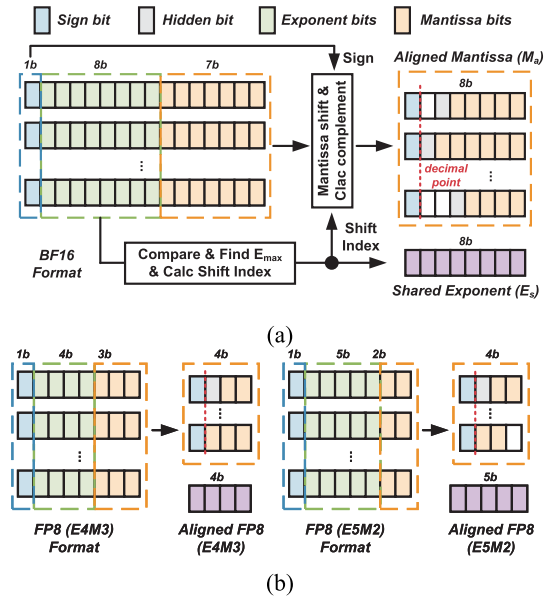


Fig. 8. (a) SFME for BF16 format. (b) SFME for FP8 E4M3 and E5M2 format.

IV. MANTISSA ENCODE AND PRE-ALIGNMENT

A. Signed Fixed-Point Mantissa Encode

Most of the prior FP CIM works primarily support the BF16 format [5], [10], [19], [25] and FP16 format [20], [26]. The FP8 format has been proposed as a novel FP representation to enable more energy-efficient training and inference [27], and it is already being adopted in state-of-the-art large language model training [28]. The FP8 format can be represented in two distinct configurations: 5-bit exponent with 2-bit mantissa (E5M2) and 4-bit exponent with 3-bit mantissa (E4M3). The E5M2 format offers a larger dynamic range than E4M3 at the expense of lower precision, whereas E4M3 provides higher precision than E5M2 but with a reduced dynamic range.

To support both the high-precision BF16 format and the energy-efficient FP8 format, as well as commonly used INT quantization formats such as INT4 and INT8, this work

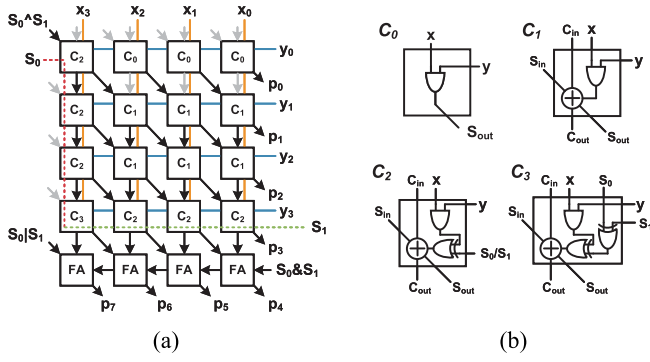


Fig. 9. (a) Structure of the proposed quad-mode multiplier. (b) PEs of the proposed quad-mode multiplier.

proposes the SFME. As illustrated in Fig. 8(a), the SFME first identifies the maximum exponent E_{max} by comparing all exponents of the FP numbers to be aligned. The differences between each exponent and the maximum exponent are then computed as shift indexes $SI_i = E_{max} - E_i$. Subsequently, the mantissas along with the hidden bits of all FP numbers are right-shifted according to their respective shift indexes and converted into two's complement format based on their sign bits. Finally, the sign bit and the two's complement mantissa are combined to form a signed fixed-point number, with the decimal point placed behind the sign bit. Following the SFME processing, the original BF16 numbers are transformed into 8-bit aligned mantissas M_a and a shared 8-bit exponent E_s . As shown in Fig. 8(b), the SFME also supports format conversion for both FP8 E5M2 and E4M3 representations. Specifically, FP8 E5M2 numbers are converted into 4-bit aligned mantissas and a shared 4-bit exponent, whereas FP8 E4M3 numbers are transformed into 4-bit aligned mantissas and a shared 5-bit exponent. The SFME can be represented as the following formula:

$$\begin{aligned} FP &= (-1)^S \times 2^{E-Bias} \times (1.M) \\ &= 2^{E_{max}-Bias} \times [(-1)^S \times (0.M_n)] \\ &= 2^{E_s} \times M_a. \end{aligned} \quad (1)$$

To enable a unified hardware implementation of the SFME across different formats, a 4-bit quad-mode multiplier is proposed. This design is based on the following formula, which allows 8-bit multiplication to be decomposed into multiple 4-bit operations:

$$\begin{aligned} (W_{[7:4]} \ll 4 + W_{[3:0]}) \times (I_{[7:4]} \ll 4 + I_{[3:0]}) \\ = (W_{[7:4]} \times I_{[7:4]}) \ll 8 + (W_{[3:0]} \times I_{[7:4]}) \ll 4 \\ + (W_{[7:4]} \times I_{[3:0]}) \ll 4 + (W_{[3:0]} \times I_{[3:0]}). \end{aligned} \quad (2)$$

In the formula, for signed 8-bit multiplication, the four 4-bit partial products are categorized into four distinct cases: signed–signed, unsigned–signed, signed–unsigned, and unsigned–unsigned multiplication. The architecture of the proposed quad-mode multiplier is illustrated in Fig. 9(a). This design is inspired by the Baugh–Wooley multiplier architecture [29] and has been further modified to support all four multiplication scenarios. The four types of processing elements (PEs) that constitute the multiplier are shown in Fig. 9(b).

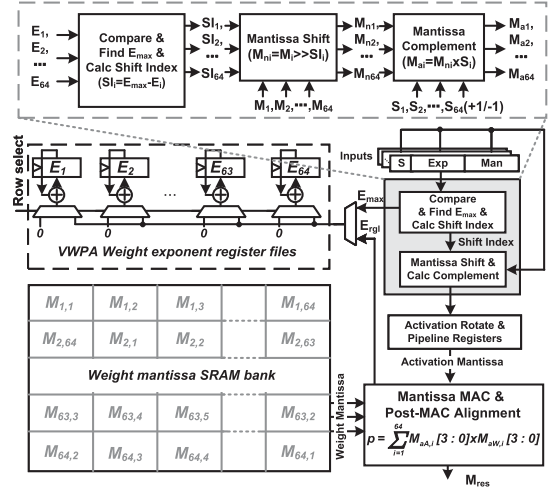


Fig. 10. Proposed VWPA scheme.

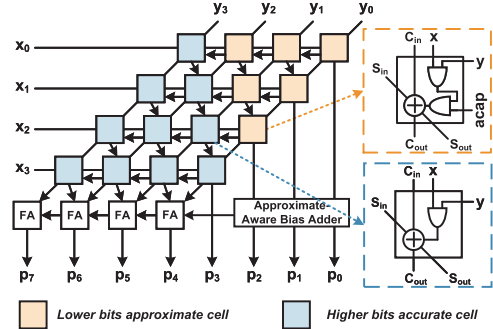


Fig. 11. Accurate/approximate DM-MUL architecture.

B. Vector-Wise Pre-Alignment

To achieve lower accuracy loss, this work proposes the VWPA scheme. As illustrated in Fig. 10, the overall architecture is inspired by previous pre-alignment FP CIM designs. However, to enable fine-grained pre-alignment and thereby further reduce accuracy degradation, we perform the pre-alignment of FP weights vector-wise rather than layer-wise, as used in prior FP CIM implementations [5]. To support this design, we incorporate 64 VWPA weight exponent register files. The pre-aligned shared exponents of the weights are stored in these register files and are combined with the shared exponents of the input FP activations and the post-MAC alignment regular exponent values. The accuracy improvement achieved by VWPA will be analyzed in detail in Section IV-A.

V. DUAL-MODE BIT-PARALLEL MAC CIRCUITS

A. DM-ADD and Multiplier

Approximate computing is a promising approach for achieving high energy efficiency by trading off a certain degree of computational accuracy [31], [32] and has been widely adopted in prior CIM macros [7], [8]. To enable a flexible tradeoff between accuracy and energy efficiency across different application scenarios, this work proposes the DMBP-MAC scheme. The main components of the proposed DMBP-MAC are illustrated in Figs. 11 and 12, which feature the dual-mode multiplier (DM-MUL) and dual-mode

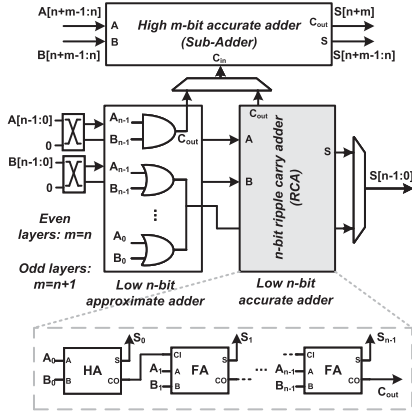


Fig. 12. Accurate/approximate DM-ADD architecture.

adder (DM-ADD), respectively. The DM-MUL further modified previously mentioned quad-mode multiplier based on a broken-array multiplier architecture that discards the lower six partial products, while the DM-ADD is implemented using a lower part or adder (LOA), in which the lower stage full adders are replaced with OR gates [33].

To evaluate the impact of the approximate hardware on computing accuracy, 50 000 random input samples are tested. As shown in Fig. 13(a), the measured mean relative error distance (MRED) is 5.3%, which can be seen as a moderate error range [32]. To further analyze the characteristics of the approximation error, the error distribution is plotted in Fig. 13(b), which reveals a Gaussian distribution with a mean close to zero, indicating that the approximation introduces minimal systematic bias.

The performance improvements achieved by approximate computing are also evaluated. Taking the accurate mode as the baseline, under a supply voltage of 0.9 V and with INT4 input and weight operands, the approximate mode demonstrates a 12% reduction in latency and a 45% reduction in power consumption under identical activation and weight settings. The DM-MUL incurs a 3.6% area overhead over the single-mode baseline, while the DM-ADD requires 17.9% additional area. Overall, the DMBP-MAC circuit contributes only 3.1% additional area to the CIM macro, which is a negligible penalty compared to energy efficiency improvement.

B. Centrosymmetric CIM Layout

As shown in Fig. 14, a centrosymmetric layout is proposed to tightly integrate the DMBP-MAC with the CWM-SRAM sub-banks. The entire layout is organized into five hierarchical levels. The first-level CIM block consists of two CWM-SRAM sub-banks, two DM-MULs, and one 8-bit DM-ADD. Two level-1 CIM blocks are symmetrically arranged to form a level-2 CIM block. The CIM blocks from level 3 to level 5 are constructed in the same manner, following this hierarchical and symmetric integration strategy. During computation, all sub-banks are simultaneously activated to deliver the weight vector to the DMBP-MAC. In contrast, during a standard SRAM read operation, only the selected sub-banks are activated, and their

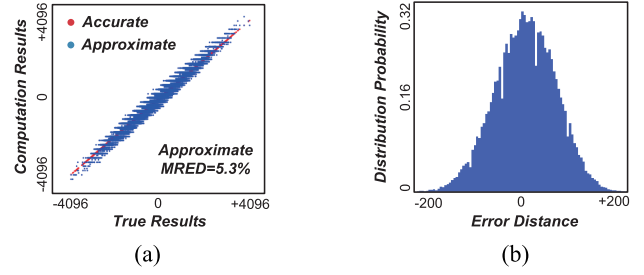


Fig. 13. (a) MRED of approximate mode. (b) Distribution of approximate error.

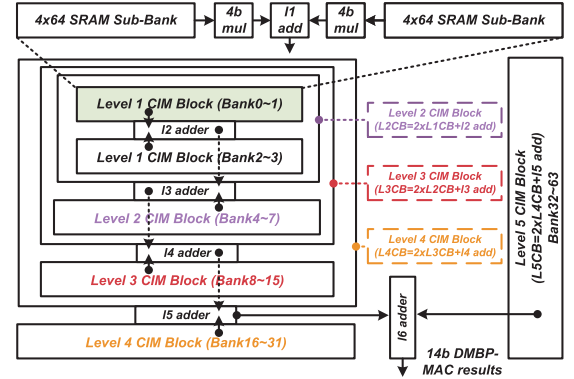


Fig. 14. Centrosymmetric layout for DMBP-MAC and CWM-SRAM.

outputs are gathered by a central readout multiplexer within the CIM macro.

Compared to previous digital CIM layout designs [22], [24], where adders are placed and routed parallel to the readout direction of the SRAM sub-banks, this work adopts a vertical layout and routing strategy. This choice is based on the observation that the proposed digital CIM employs a 64-row number to enlarge memory density, significantly deeper than prior works that typically use only one or two rows. A parallel layout and routing approach would substantially increase routing length and lead to routing congestion. This is particularly problematic given that metal layers M3–M5 are already occupied by control signals for FF and BP in the CWM-SRAM sub-banks.

VI. ANALYSIS AND MEASUREMENT RESULTS

A. Analysis of the Proposed CIM Macro

As illustrated in Fig. 15, to evaluate the tradeoff between accuracy and energy efficiency across different numerical data formats, both the top-1 accuracy degradation on the CIFAR-100 dataset using the VGG-19 model and the average energy efficiency under a 0.5-V supply voltage are analyzed for INT4, INT8, FP8 (E4M3), and BF16, considering both accurate and approximate computation modes. Compared to commonly adopted formats such as INT8 and BF16 in prior works, FP8 achieves nearly three times higher energy efficiency with only a marginal decrease in accuracy. Although INT4 demonstrates even greater energy efficiency, its significant accuracy degradation limits its practical applicability.

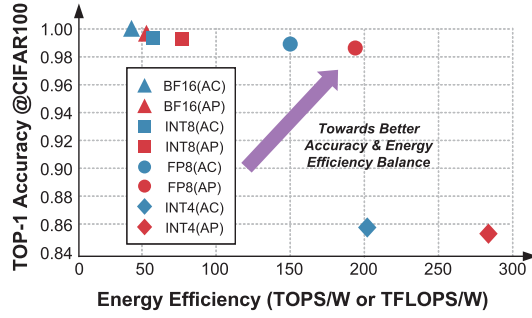


Fig. 15. Comparison of accuracy and energy efficiency for different formats.

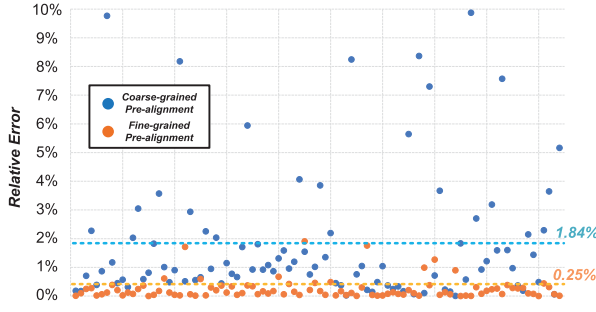


Fig. 16. Comparison of coarse-grained pre-alignment and fine-grained pre-alignment.

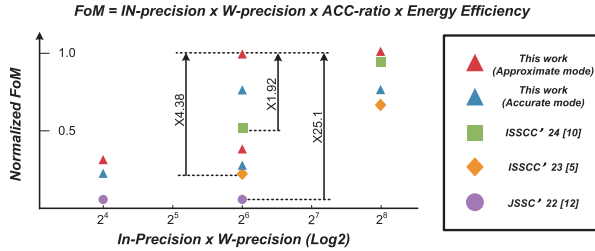


Fig. 17. Comparison of FoM with previous works.

Therefore, FP8 strikes a more favorable balance between accuracy and energy efficiency, making it a promising candidate for adoption in future energy-efficient hardware architectures.

To justify the effectiveness of VWPA, 100 dot-product results from randomly generated matrix-vector multiplication operations were simulated and sampled using both coarse-grained pre-alignment and VWPA fine-grained pre-alignment. The resulting relative errors were computed with respect to a software baseline that does not employ any alignment techniques, as illustrated in Fig. 16. The coarse-grained pre-alignment yields an average relative error of 1.84%, whereas the fine-grained pre-alignment reduces the average error to only 0.25%. Furthermore, the maximum relative error under coarse-grained pre-alignment can approach nearly 10%, while the fine-grained approach consistently maintains relative errors below 2%.

To comprehensively evaluate the advantages of the proposed CIM macro, we benchmarked this work against three prior studies using a figure of merit (FoM), defined as the product of input precision, weight precision, accuracy ratio,

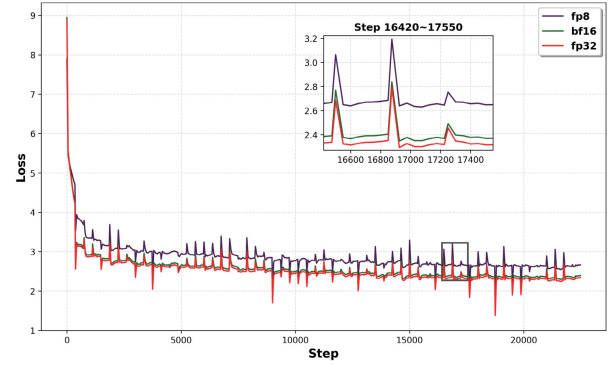


Fig. 18. Training loss curve with the proposed methods.

and energy efficiency. In our calculation, BF16 is treated as 16-bit precision, and FP8 as 8-bit precision. The accuracy ratio is defined as the ratio of the actual output accuracy to the ideal output accuracy. All works are evaluated based on their maximum average energy efficiency and accuracy achieved on the CIFAR-100 dataset. As shown in Fig. 17, our design achieves a significantly higher FoM compared to previous works. Notably, in FP8 mode, our FoM ranges from 1.92 to 25.1 times higher than that of prior works. To validate training accuracy and stability with the proposed methods, we implemented an end-to-end training framework for the proposed CIM macro. To realize training with the proposed CIM macro, we first initialize it with random weights and operate in FF mode. After each FF operation, the output activations are read out to the host processor for loss calculation and gradient computation. The CIM macro is then switched to BP mode, and the gradients are sent back to update the weights. This FF-BP cycle is repeated until training converges. Evaluated on ViT-S trained on ImageNet, the BF16 mode exhibits a nearly identical loss curve to the FP32 baseline, while the FP8 mode shows greater accuracy degradation but still achieves training convergence, as shown in Fig. 18.

Table I presents a comparative analysis between the proposed CIM macro and prior works. Compared to prior non-CIM digital accelerators—which relied on two-way readable register files to perform transposition for backward data paths [30], CIM designs eliminate the need for data movement by coupling MAC circuits directly with memory, enabling computation at the storage location. Compared to previous transpose CIM implementations, this work achieves notable improvements in both energy efficiency and area efficiency. These enhancements are primarily attributed to the reuse of the CWM-SRAM MAC circuits across the forward and backward propagation phases. By incorporating bit-parallel and approximate arithmetic techniques in the DMBP-MAC, the proposed design achieves shorter access times, as well as higher energy and area efficiencies, compared to earlier non-transpose CIM implementations. Furthermore, with the proposed VWPA technique, the macro demonstrates lower accuracy loss in accurate mode when compared to layer-wise pre-aligned FP CIM approaches. Thanks to the increased number of rows in the SRAM sub-banks, the memory density of the proposed CIM macro is also improved over prior works.

TABLE I
PERFORMANCE COMPARISON WITH PRIOR WORKS

	JSSC'22 [12]	ASSC'21 [13]	ISSCC'23 [5]	ISSCC'25 [30]	This Work	
	J.-W. Su	J.Song	A.Guo	J.Heo	Approx.	Accurate
Technology	28nm	28nm	28nm	14nm	28nm	
CIM Type	Analog Transpose CIM	Analog Transpose CIM	Digital Non-transpose CIM	Digital Transpose Non-CIM	Digital Transpose CIM	
Precision	INT2/4/8	INT1	INT8,BF16	INT12,FP12	INT4/8,FP8,BF16	
Access Time	15.2ns @INT8	–	6.8ns @BF16	–	2.5ns @FP8 5.1ns @BF16	2.7ns @FP8 5.4ns @BF16
Memory Density	–	0.273Mb/mm ²	0.428Mb/mm ²	–	0.732Mb/mm ²	
Area Efficiency	–	0.056 TOPS/mm ²	2.57 TOPS/mm ²	0.56 TOPS/mm ²	3.23 TOPS/mm ²	2.89 TOPS/mm ²
			2.05 TFLOPS/mm ²	0.56 TFLOPS/mm ²	2.34 TFLOPS/mm ²	2.24 TFLOPS/mm ²
			19.5-44 TOPS/W @INT8	8.9-11.0 TOPS/W @INT12	25.5-71.4 TOPS/W @INT8	17.54-50 TOPS/W @INT8
INT Energy Eff.	7-7.6 TOPS/W @INT8	4 TOPS/W @INT8	14.04-31.6 TFLOPS/W @BF16	12.3-12.8 TFLOPS/W @FP12	17.19-48.08 TFLOPS/W @BF16	12.78-36.44 TFLOPS/W @BF16
FP Energy Eff.	–	–	0.17% @CIFAR100	–	0.05% @CIFAR10 0.29% @CIFAR100	0.01% @CIFAR10 0.1% @CIFAR100
Accuracy Loss	2.27% @CIFAR100	2.98% @CIFAR10	0.17% @CIFAR100	–	0.05% @CIFAR10 0.29% @CIFAR100	0.01% @CIFAR10 0.1% @CIFAR100

TABLE II
FF AND BP COMPARISON

Direction	Feed-Forward				Back-Propagation	
Input Precision	INT4	INT8	FP8	BF16	FP8	BF16
Weight Precision	INT4	INT8	FP8	BF16	FP8	BF16
Average Energy Efficiency	Approx.	102 TOPS/W	25.5 TOPS/W	68.8 TFLOPS/W	17.2 TFLOPS/W	–
	Accurate	70.2 TOPS/W	17.5 TOPS/W	51.1 TFLOPS/W	12.8 TFLOPS/W	49.9 TFLOPS/W

Table II presents energy efficiency comparison between FF and BP operations under 0.9 V, 50% sparsity, and 50% toggle rate condition. A key distinction is that, to ensure training stability, only FP8/BF16 precisions and the accurate mode are employed for BP. The average EF of BP exhibits a slight degradation compared to FF, which can be attributed to the longer routing overhead in the CWM-SRAM. To evaluate the typical energy efficiency of the proposed CIM macro under realistic model sparsity and toggle rate conditions, we perform both inference and training of a ViT-S model on ImageNet. We use an approximate FP8 configuration for inference and a high-precision BF16 configuration for training. The measured energy efficiency is 65.2 TFLOPS/W for inference and 10.5 TFLOPS/W for training. We focus exclusively on the energy efficiency of the CIM macro itself. Therefore, energy consumption associated with off-chip data movement and on-chip control circuits is excluded.

B. Analysis of System-Level Applications

Most prior CIM works have focused exclusively on macro-level performance optimization, while critical system-level integration challenges have remained overlooked for years. We mainly focus on two key challenges. First is the scalability bottleneck, while CIM macros demonstrate high efficiency for small-scale matrix multiplication, our analysis reveals

fundamental physical barriers to monolithic scaling, as array size increases, parasitic capacitance in global BLs and limited driver strength in peripheral circuits cause 33% energy efficiency degradation penalty when our CIM macro scaled to 64 kB. Thus, rather than pursuing impractical single-macro scaling, we suggest a modular expansion strategy by treating each CIM macro as a reconfigurable PE.

Second is the software-to-hardware mapping gap. Developing a full software stack for CIM from scratch is prohibitively costly and time-consuming. To address this challenge, we leverage the open-source RISC-V software ecosystem to rapidly prototype the essential operator library required for our evaluation. This integration significantly reduces development time and engineering effort for execution of NN workloads on our CIM hardware. We hope that our work can bring insight for more designs that bridge the gap between circuit-level innovation and system-level deployment.

C. Measurement Results

Fig. 19 presents the measured energy efficiency of FP8 (E4M3) in accurate and approximate computation modes. All measurements are conducted under a 50% weight sparsity condition, with input toggle rates ranging from 10% to 90%. At a 50% toggle rate, the accurate mode achieves an energy efficiency of 51.1 TFLOPS/W

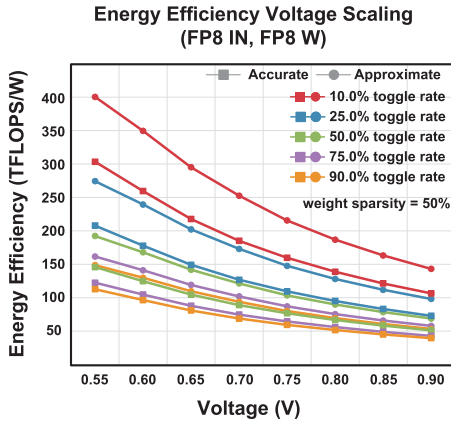


Fig. 19. Energy efficiency scaling curves of FP8 accurate and approximate modes.

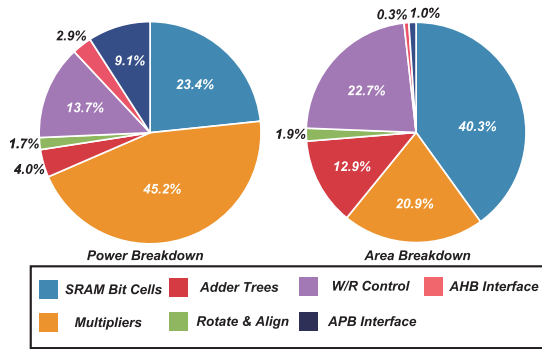


Fig. 20. Power breakdown and area breakdown.

at 0.9-V supply voltage and 145.8 TFLOPS/W at 0.55 V. In contrast, the approximate mode demonstrates improved energy efficiency, reaching 68.8 TFLOPS/W at 0.9 V and 192.3 TFLOPS/W at 0.55 V. This corresponds to an approximate 33% improvement in energy efficiency compared to the accurate mode. Under the 10% toggle rate condition, which represents peak energy efficiency, the accurate mode achieves 106.4 TFLOPS/W at 0.9 V and 303.4 TFLOPS/W at 0.55 V. The approximate mode further enhances the performance, achieving 143.0 TFLOPS/W at 0.9 V and 400.0 TFLOPS/W at 0.55 V.

Fig. 20 presents the measured power and area breakdown of the proposed CIM macro under a 0.9-V supply voltage, with 50% weight sparsity and 50% input toggle rate in INT4 accurate mode. In terms of power distribution, the multipliers account for the largest share, contributing 45.3% of the total power consumption. The adder tree consumes only 4.0%, while the SRAM arrays and control circuits account for 23.4% and 13.7%, respectively. The rotation and alignment circuits contribute a relatively minor portion, consuming 1.7% of the total power. The AHB and APB interfaces consume 2.9% and 9.1%, respectively. Regarding the area breakdown, the SRAM arrays and control circuits are the dominant components, occupying 40.3% and 22.7% of the total area, respectively. The multipliers and adder trees take up 20.9% and 12.9%, while the rotate and alignment circuits occupy only 1.9% of the area. The AHB and APB interfaces occupy 0.3% and 1.0% of the total area, respectively. Unlike bit-serial CIM macros,

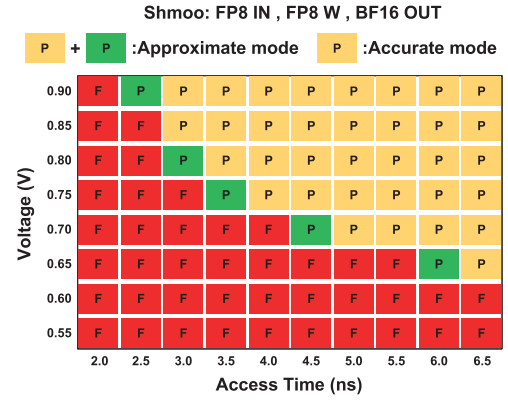


Fig. 21. Shmoo plots of FP8 accurate and approximate modes.

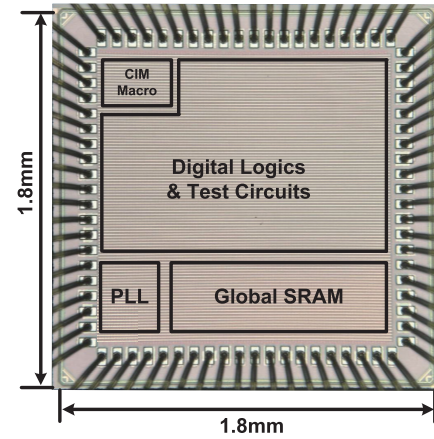


Fig. 22. Die photograph of the proposed CIM macro.

where the adder tree typically forms the power bottleneck, the power bottleneck in the bit-parallel CIM macro stems from the large number of high-power-consuming multipliers. This characteristic highlights the need for future optimization efforts to focus on multiplier energy efficiency. Furthermore, although a significant portion of the chip area is allocated to SRAM, the macro achieves favorable area efficiency due to the throughput improvements enabled by the bit-parallel architecture.

Fig. 21 presents the measured shmoo plots of the proposed CIM macro operating in FP8-IN, FP8-W, BF16-OUT accurate, and approximate modes. The shortest access times are 2.5 ns at $V_{DD} = 0.9$ V in the accurate mode and 2.7 ns at $V_{DD} = 0.9$ V in the approximate mode. Fig. 22 shows a die microphotograph of the fabricated chip, and Fig. 23 illustrates the measurement setup. The chip, implemented in a 28-nm process, occupies an area of 1.8×1.8 mm and integrates the proposed CIM macro along with a RISC-V SoC to support system-level application development. The measurement system consists of a DC power supply, an oscilloscope, and a test board connected to a PC via a JTAG debugger.

Table III summarizes the key characteristics of the proposed CIM macro. The macro features a capacity of 32 kB and operates across a supply voltage range of 0.55–0.9 V, with a total area of 0.043 mm^2 . We evaluated the performance of the CIM macro in terms of access time, average and peak energy

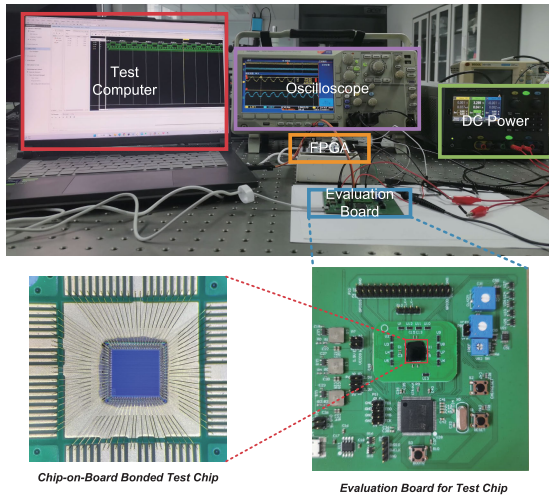


Fig. 23. Measurement system of the proposed CIM macro.

TABLE III
CHIP SUMMARY

Technology	28nm			
Macro Area	0.043mm ² (253.8x168.3μm)			
Macro Size	32Kb			
Supply Voltage	0.55-0.9V			
Channel Number	128			
Input Precision	INT4	INT8	FP8	BF16
Weight Precision	INT4	INT8	FP8	BF16
Access Time				
AP @0.9V	0.9ns	1.9ns	2.5ns	5.1ns
AC @0.9V	1.0ns	2.1ns	2.7ns	5.3ns
Average EF				
AP @50% TR	102-285.4 TOPS/W	25.5-71.4 TOPS/W	68.8-192.3 TFLOPS/W	17.2-48 TFLOPS/W
AC @50% TR	70.2-200 TOPS/W	17.5-50 TOPS/W	51.1-145.8 TFLOPS/W	12.8-36.4 TFLOPS/W
Peak EF				
AP @10% TR	212.4-594 TOPS/W	53-148.5 TOPS/W	143-400 TFLOPS/W	35.8-100 TFLOPS/W
AC @10% TR	146-416.5 TOPS/W	36.5-104 TOPS/W	106.4-303.4 TFLOPS/W	26.6-75.8 TFLOPS/W
Accuracy Loss				
AP @CIFAR10	3.70%	0.12%	0.42%	0.05%
AC @CIFAR10	3.62%	0.04%	0.40%	0.01%
AP @CIFAR100	14.21%	0.41%	1.24%	0.29%
AC @CIFAR100	14.13%	0.30%	1.00%	0.10%
AP @ImageNet	29.96%	0.68%	1.78%	0.60%
AC @ImageNet	27.88%	0.26%	1.33%	0.03%

efficiency, and accuracy loss using three benchmark models: ResNet-18 on CIFAR-10, VGG-19 on CIFAR-100, and ViT-S on ImageNet. During evaluation, the fully connected layers, convolution layers, and matrix multiplications of attention layers are performed with the proposed CIM macro. The accuracy loss exhibits a clear trend. Although the accuracy gap between the accurate and approximate modes widens with larger datasets and more complex models, the degradation remains acceptable, staying below 1% in most of the cases.

VII. CONCLUSION

To conclude, this work presents a digital transpose SRAM CIM macro supporting INT4/8, FP8, and BF16 formats.

To enable energy-efficient and privacy-protected edge training, a transpose-supported SRAM CIM macro is essential. However, previous transpose SRAM CIM macros are restricted by redundant MAC circuits, limited format support, and PVT variations. To solve these challenges, this work features a CWM-SRAM to reuse MAC circuits for FF and BP phases and therefore improves energy and area efficiency; SFME to support varied INT/FP formats and VWPA to further improve accuracy; Finally, DMBP-MAC is introduced to trade off between accuracy and extremely high energy and area efficiency. The fabricated 28-nm 32-kB transpose SRAM CIM macro achieved average energy efficiency of 70.2–285.4 TOPS/W in INT4, 17.5–71.4 TOPS/W in INT8, 51.1–192.3 TFLOPS/W in FP8, and 12.8–48 TFLOPS/W in BF16.

REFERENCES

- [1] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and memory wall," *IEEE Micro*, vol. 44, no. 3, pp. 33–39, May 2024.
- [2] M. Ali, S. Roy, U. Saxena, T. Sharma, A. Raghunathan, and K. Roy, "Compute-in-memory technologies and architectures for deep learning workloads," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 11, pp. 1615–1630, Nov. 2022.
- [3] T. Sharma, M. Ali, I. Chakraborty, and K. Roy, "WWW: What, when, where to compute-in-memory," 2025, *arXiv:2312.15896*.
- [4] S. Yu, H. Jiang, S. Huang, X. Peng, and A. Lu, "Compute-in-memory chips for deep learning: Recent trends and prospects," *IEEE Circuits Syst. Mag.*, vol. 21, no. 3, pp. 31–56, 3rd Quart., 2021.
- [5] A. Guo et al., "A 28nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 128–130.
- [6] H. Fujiwara et al., "A 3nm, 32.5TOPS/W, 55.0TOPS/mm² and 3.78Mb/mm² fully-digital compute-in-memory macro supporting INT12 × INT12 with a parallel-MAC architecture and foundry 6T-SRAM bit cell," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 572–574.
- [7] Y. He et al., "A 28nm 38-to-102-TOPS/W 8b multiply-less approximate digital SRAM compute-in-memory macro for neural-network inference," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2023, pp. 130–132.
- [8] D. Wang, C.-T. Lin, G. K. Chen, P. Knag, R. K. Krishnamurthy, and M. Seok, "DIMC: 2219TOPS/W 2569F2/b digital in-memory computing macro in 28nm based on approximate arithmetic hardware," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 266–268.
- [9] Y. Wang et al., "A 28 nm 83.23TFLOPS/W POSIT-based compute-in-memory macro for high-accuracy AI applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 566–568.
- [10] W.-S. Khwa et al., "A 16nm 96Kb integer/floating-point dual-mode-gain-cell-computing-in-memory macro achieving 73.3–163.3TOPS/W and 33.2–91.2TFLOPS/W for AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 568–570.
- [11] S. Zhu, T. Voigt, F. Rahimian, and J. Ko, "On-device training: A first overview on existing systems," *ACM Trans. Sensor Netw.*, vol. 20, no. 6, pp. 1–39, Oct. 2024.
- [12] J.-W. Su et al., "Two-way transpose multibit 6T SRAM computing-in-memory macro for inference-training AI edge chips," *IEEE J. Solid-State Circuits*, vol. 57, no. 2, pp. 609–624, Feb. 2022.
- [13] J. Song, Y. Wang, X. Tang, R. Wang, and R. Huang, "A 16Kb transpose 6T SRAM in-memory-computing macro based on robust charge-domain computing," in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, Nov. 2021, pp. 1–3.
- [14] H. Jiang, X. Peng, S. Huang, and S. Yu, "CIMAT: A compute-in-memory architecture for on-chip training based on transpose SRAM arrays," *IEEE Trans. Comput.*, vol. 69, no. 7, pp. 944–954, Jul. 2020.
- [15] S. K. Venkataramanaiyah, S. Yin, Y. Cao, and J.-S. Seo, "Deep neural network training accelerator designs in ASIC and FPGA," in *Proc. Int. SoC Design Conf. (ISOC)*, Oct. 2020, pp. 21–22.

- [16] S. Wang and Y. Kang, "Gradient distribution-aware INT8 training for neural networks," *Neurocomputing*, vol. 541, Jul. 2023, Art. no. 126269.
- [17] H. Diao et al., "A 28nm 128TFLOPS/W computing-in-memory engine supporting one-shot floating-point NN inference and on-device fine-tuning for edge AI," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2024, pp. 1–2.
- [18] A. Guo et al., "ShareFloat CIM: A compute-in-memory architecture with floating-point multiply-and-accumulate operations," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Austin, TX, USA, May 2022, pp. 2276–2280.
- [19] F. Tu et al., "A 28 nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.
- [20] T.-H. Wen et al., "A 22nm 16Mb floating-point ReRAM compute-in-memory macro with 31.2TFLOPS/W for AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 580–582.
- [21] Y.-D. Chih et al., "An 89TOPS/W and 16.3TOPS/mm² all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 252–254.
- [22] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm² fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 65, Feb. 2022, pp. 1–3.
- [23] H. Mori et al., "A 4nm 6163-TOPS/W/b 4790 – TOPS/mm²/b SRAM based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 132–134.
- [24] C.-F. Lee et al., "A 12nm 121-TOPS/W 41.6-TOPS/mm² all digital full precision SRAM-based compute-in-memory with configurable bit-width for AI edge applications," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 24–25.
- [25] P.-C. Wu et al., "A 22nm 832Kb hybrid-domain floating-point SRAM in-memory-compute macro with 16.2–70.2TFLOPS/W for high-accuracy AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 126–128.
- [26] S. Yan et al., "A 28-nm floating-point computing-in-memory processor using intensive-CIM sparse-digital architecture," *IEEE J. Solid-State Circuits*, vol. 59, no. 8, pp. 2630–2643, Aug. 2024.
- [27] P. Micikevicius et al., "FP8 formats for deep learning," 2022, *arXiv:2209.05433*.
- [28] A. Hernández-Cano, D. Garbaya, I. Schlag, and M. Jaggi, "Towards fully FP8 GEMM LLM training at scale," 2025, *arXiv:2505.20524*.
- [29] H. El-Razouk and Z. Abid, "Area and power efficient array and tree multipliers," in *Proc. Can. Conf. Electr. Comput. Eng.*, vol. 2, 2006, pp. 713–716.
- [30] J. Heo et al., "23.10 HuMoniX: A 57.3fps 12.8TFLOPS/W text-to-motion processor with inter-iteration output sparsity and inter-frame joint similarity," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 424–426.
- [31] Q. Xu, T. Mytkowicz, and N. S. Kim, "Approximate computing: A survey," *IEEE Design Test*, vol. 33, no. 1, pp. 8–22, Feb. 2016.
- [32] V. Leon et al., "Approximate computing survey, part I: Terminology and software & hardware approximation techniques," *ACM Comput. Surveys*, vol. 57, no. 7, pp. 1–36, Mar. 2025.
- [33] H. R. Mahdiani, A. Ahmadi, S. M. Fakhraie, and C. Lucas, "Bio-inspired imprecise computational blocks for efficient VLSI implementation of soft-computing applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 57, no. 4, pp. 850–862, Apr. 2010.



Yiyang Yuan received the B.S. degree in electronic and information engineering from Beijing Institute of Technology, Beijing, China, in 2021. He is currently pursuing the Ph.D. degree with the Institute of Microelectronics of Chinese Academy of Sciences, Beijing.

His research interests include compute-in-memory, resistive random access memory, artificial intelligence, and brain-machine interface.



Bingxin Zhang received the B.S. degree in electronic science and technology from Beijing University of Chemical Technology, Beijing, China, in 2023. He is currently pursuing the M.S. degree in electronic information with the Institute of Microelectronics of Chinese Academy of Sciences, Beijing.

His current research interests include SRAM-based high-efficiency neural network accelerators, SRAM-based computing-in-memory macro design, and computing-in-memory architectures.



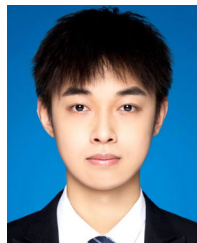
Yiming Yang received the B.S. degree in microelectronics from Beijing Institute of Technology, Beijing, China, in 2020, where he is currently pursuing the Ph.D. degree.

His research interests include compute-in-memory and RF circuits design.



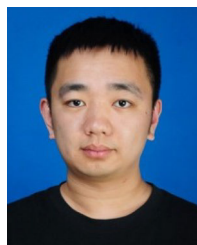
Yishan Luo received the B.E. degree in electronic science and technology from Beijing Institute of Technology, Beijing, China, in 2024. He is currently pursuing the Ph.D. degree with the Institute of Microelectronics of Chinese Academy of Sciences, Beijing.

His current research interests include SRAM-RRAM hybrid in-memory computing accelerators, in-memory computing simulators, and compilers for in-memory computing.



Qirui Chen received the B.S. degree in electronics information engineering from Beijing Institute of Technology, Beijing, China, in 2023, where he is currently pursuing the M.S. degree.

His research interests include energy-efficient architecture for machine learning algorithms and compute-in-memory system design.



Haitao Wang received the B.S. degree from Anhui University of Technology and the M.S. degree from Chongqing University of Posts and Telecommunications.

He is currently at Chongqing Institute of Microelectronics and Microsystems, Beijing Institute of Technology, Chongqing, China. His research interests focus on pattern recognition and computer vision.



Qihao Liu received the B.S. and M.S. degrees from Yangtze University, Jingzhou, China, in 2009 and 2013, respectively.

He was with a Joint-Training Program, Institute of Microelectronics of Chinese Academy of Sciences (IMECAS), Beijing, China, from 2011 to 2013. In 2013, he joined Sino-Chip Semiconductors, Jinan, China, as an SOC Architect. His research interests mainly include storage and security technology, such as advanced non-volatile memory and encryption applications.



Zhiming Chen received the B.Eng. degree in electronic science and technology from Tsinghua University, Beijing, China, in 2007, and the Ph.D. degree in electrical and computer engineering from the University of California, Irvine, Irvine, CA, USA, in 2012.

In Spring 2012, he joined Beijing Institute of Technology, Beijing, as a Faculty Member, where he is currently a Professor with the School of Integrated Circuits and Electronics. His research

interests include analog, digital, RF and millimeter-wave integrated circuits design, advanced packaging, and 3-D integrated circuits.



Hao Wu received the B.S. degree in integrated circuits and integrated systems from Xidian University, Xi'an, China. He is currently pursuing the Ph.D. degree with the Key Laboratory of Microelectronics Device and Integrated Technology, Institute of Microelectronics of Chinese Academy of Sciences, Beijing, China.

His research interests are SRAM-based in-memory computing processor design and algorithm-architecture co-design.



Jinshan Yue (Member, IEEE) received the B.S. and Ph.D. degrees from the Department of Electronic Engineering, Tsinghua University, Beijing, China, in 2016 and 2021, respectively.

From 2021 to 2024, he was a Post-Doctoral Researcher at the Institute of Microelectronics of Chinese Academy of Sciences (IMECAS), Beijing. He is currently an Associate Researcher with IMECAS. His current research interests include energy-efficient neural network processors, non-volatile memory, and computing-in-memory systems.

Dr. Yue was a TPC Member of IEEE A-SSCC and ICCAD.



Shidong Lv received the B.Eng. and M.E. degrees from Beijing Institute of Technology (BIT), Beijing, China, in 2004 and 2006, respectively, where he is currently pursuing the Ph.D. degree with the School of Integrated Circuits and Electronics.

His research interests include RF and signal processing.



Xinghua Wang received the B.Eng. and Ph.D. degrees from Beijing Institute of Technology (BIT), Beijing, China, in 2004 and 2010, respectively.

She is currently an Associate Professor at the School of Integrated Circuits and Electronics, BIT. Her research interests include analog, RF, and computing-in-memory.



Pui-In Mak (Fellow, IEEE) received the B.Sc. and Ph.D. degrees from the University of Macau (UM), Macau, China, in 2003 and 2006, respectively.

He is currently a Professor with the Faculty of Science and Technology-ECE, the Director of the State-Key Laboratory of Analog and Mixed-Signal VLSI, and the Deputy Director (Research) of the Institute of Microelectronics, UM. His research interests include analog and radio frequency (RF) circuits and systems for wireless and multidisciplinary innovations.

Dr. Mak was a TPC Member of the IEEE International Solid-State Circuits Conference (ISSCC), A-SSCC, and ESSCIRC. He is a fellow of IET and U.K. Royal Society of Chemistry (RSC). He received the Tencent Xplorer Prize in 2022 and was recognized in 2023 as one of the Top Contributing Authors of ISSCC in its 70 years of history. He is currently the Editor-in-Chief of IEEE SOLID-STATE CIRCUITS LETTERS. He is an Overseas Expert of Chinese Academy of Sciences. He is named as a Foreign Academician of the Portuguese Academy of Sciences in 2024. He was a Distinguished Lecturer of the IEEE Circuits and Systems Society and the IEEE Solid-State Circuits Society. His team reported 39 state-of-the-art chips from 2011 to 2024 ISSCC.



Xiaoran Li received the B.Eng. degree in telecommunication engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2011, and the M.E. and Ph.D. degrees in electronics science and technology from Beijing Institute of Technology (BIT), Beijing, in 2013 and 2017, respectively.

She was a Visiting Scholar with the Department of Electrical and Computer, Duke University, Durham, NC, USA, from 2015 to 2016. She was a Post-Doctoral Research Fellow at BIT, where she is

currently an Assistant Professor. Her research interests include analog, radio frequency, millimeter-wave integrated circuits design, computing in memory, sparse sensing, and communications.



Feng Zhang (Senior Member, IEEE) received the B.S. degree from Beijing Institute of Technology, Beijing, China, in 2000, and the M.S. and Ph.D. degrees from the Institute of Microelectronics of Chinese Academy of Sciences (IMECAS), Beijing, in 2002 and 2005, respectively.

He once worked on the memory design of the Loongson CPU at the Institute of Computing Technology of Chinese Academy of Sciences (ICTCAS) from 2005 to 2010. In 2010, he joined IMECAS as a Professor. He has published more than 80 international journal articles and conference papers. His research interests include memory, low-power digital circuits, and related hardware security techniques when applied to the "smart Internet of Things (IoT)" field.

Dr. Zhang serves as a Technical Committee Member for the IEEE Circuits and Systems Society (CASS).